



SW Pilot 학습보고서 <과정 9>

파이썬 5팀 황문규

문제 1. 식물 분류 프로젝트 - 코드



```
1 # 과정 9 - (문제1) 식물 분류 프로젝트
2 from sklearn.datasets import load_iris
3 import matplotlib.pyplot as plt
4
5 # [보너스 과제] 데이터 분포 시각화
6 def plot_iris_scatter(data, target, target_names):
7     plt.figure(figsize=(8, 6))
8     for class_idx, class_label in enumerate(target_names):
9         xs = data[target == class_idx, 0] # sepal length
10        ys = data[target == class_idx, 1] # sepal width
11        plt.scatter(xs, ys, label=class_label)
12
13    plt.xlabel('sepal length (cm)')
14    plt.ylabel('sepal width (cm)')
15    plt.title('Iris Dataset - Sepal Length vs Sepal Width')
16    plt.legend(title='target')
17    plt.tight_layout()
18    plt.show()
```



```
1 def main():
2     iris_dataset = load_iris()
3
4     print('DESCR:')
5     print(iris_dataset['DESCR'])
6
7     print('\nTarget Names:')
8     print(iris_dataset['target_names'])
9
10    print('\nFeature Names:')
11    print(iris_dataset['feature_names'])
12
13    data = iris_dataset['data']
14    print('\nData Info:')
15    print('Shape:', data.shape)
16    print('Dimensions:', data.ndim)
17    print('Type:', type(data))
18    print('First 5 samples:')
19    for sample in data[:5]:
20        print(sample)
21
22    target = iris_dataset['target']
23    print('\nTarget Info:')
24    print('Shape:', target.shape)
25    print('Dimensions:', target.ndim)
26    print('Type:', type(target))
27    print('First 5 targets:', target[:5])
28
29    target_names = iris_dataset['target_names']
30
31    # [보너스 과제] 데이터 분포 시각화
32    plot_iris_scatter(data, target, target_names)
33
34 if __name__ == '__main__':
35     main()
```

문제 1. 식물 분류 프로젝트 - 구현화면

```
9-1 - python main.py - 138x64
[swplot] hwangmungyu@hwangmungyu-MacBookPro ~ % cd Desktop/swplotPyhon5/src/Process9/9-1
[swplot] hwangmungyu@hwangmungyu-MacBookPro 9-1 % python main.py
DESCR:
.. _iris_dataset:

Iris plants dataset
=====
**Data Set Characteristics:**
.:Number of Instances: 150 (50 in each of three classes)
.:Number of Attributes: 4 numeric, predictive attributes and the class
.:Attribute Information:
  - sepal length in cm
  - sepal width in cm
  - petal length in cm
  - petal width in cm
  - class:
    - Iris-Setosa
    - Iris-Versicolour
    - Iris-Virginica
.:Summary Statistics:
=====
               Min   Max   Mean   SD   Class Correlation
=====
sepal length:  4.3  7.9   5.84   0.83   0.7826
sepal width:   2.0  4.4   3.05   0.43  -0.4194
petal length:  1.0  6.9   3.76   1.76   0.9448 (high)
petal width:   0.1  2.5   1.29   0.76   0.9565 (high)
=====
.:Missing Attribute Values: None
.:Class Distribution: 33.3% for each of 3 classes.
.:Creator: R.A. Fisher
.:Donor: Michael Marshall (MARSHALLNPLU@io.arc.nasa.gov)
.:Date: July, 1988

The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken
from Fisher's paper. Note that it's the same as in R, but not as in the UCI
Machine Learning Repository, which has two wrong data points.

This is perhaps the best known database to be found in the
pattern recognition literature. Fisher's paper is a classic in the field and
is referenced frequently to this day. (See Duda & Hart, for example.) The
data set contains 3 classes of 50 instances each, where each class refers to a
type of iris plant. One class is linearly separable from the other 2; the
latter are NOT linearly separable from each other.

.. dropdown: References
- Fisher, R.A. "The use of multiple measurements in taxonomic problems"
  Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to
  Mathematical Statistics" (John Wiley, NY, 1950).
- Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis.
  (0227-063) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.
- Dasarthy, B.V. (1988) "Nosing Around the Neighborhood: A New System
  Structure and Classification Rule for Recognition in Partially Exposed
  Environments". IEEE Transactions on Pattern Analysis and Machine
  Intelligence, Vol. PAMI-10, No. 3, 67-71.
- Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE Transactions
  on Information Theory, May 1972, 431-433.
- See also: 1985 MLC Proceedings, 54-64. Cheeseman et al's AUTOCLASS II
```

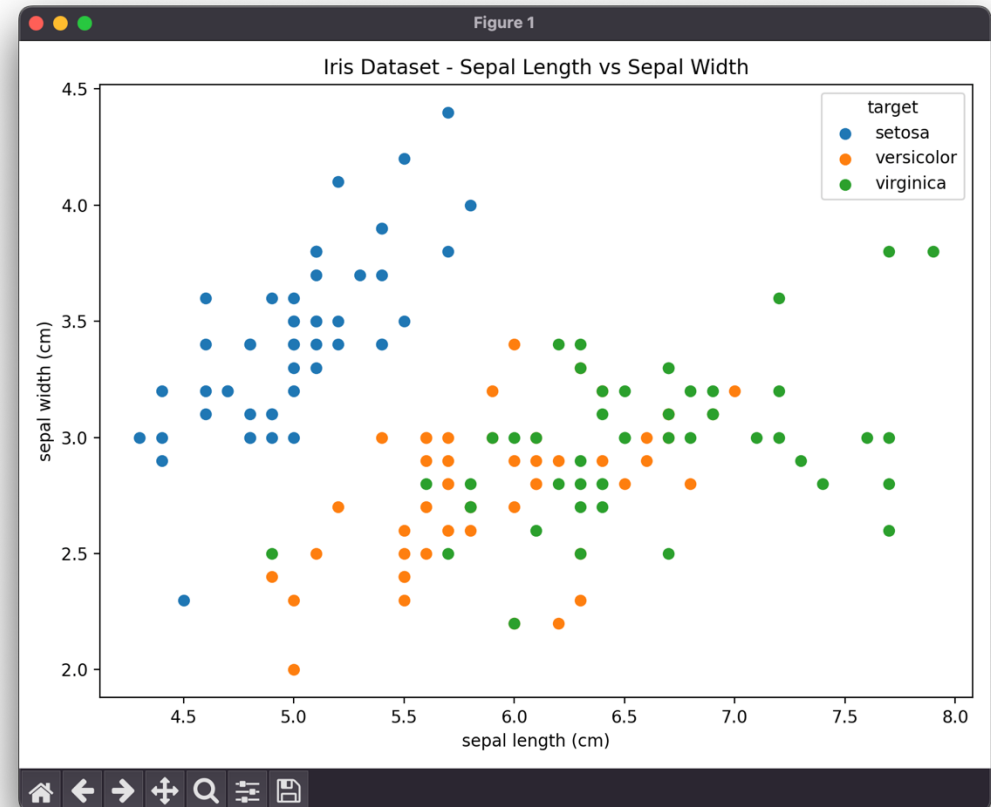
```
9-1 - python main.py - 126x24

Target Names:
['setosa' 'versicolor' 'virginica']

Feature Names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

Data Info:
Shape: (150, 4)
Dimensions: 2
Type: <class 'numpy.ndarray'>
First 5 samples:
[5.1 3.5 1.4 0.2]
[4.9 3.   1.4 0.2]
[4.7 3.2 1.3 0.2]
[4.6 3.1 1.5 0.2]
[5.   3.6 1.4 0.2]

Target Info:
Shape: (150,)
Dimensions: 1
Type: <class 'numpy.ndarray'>
First 5 targets: [0 0 0 0 0]
```

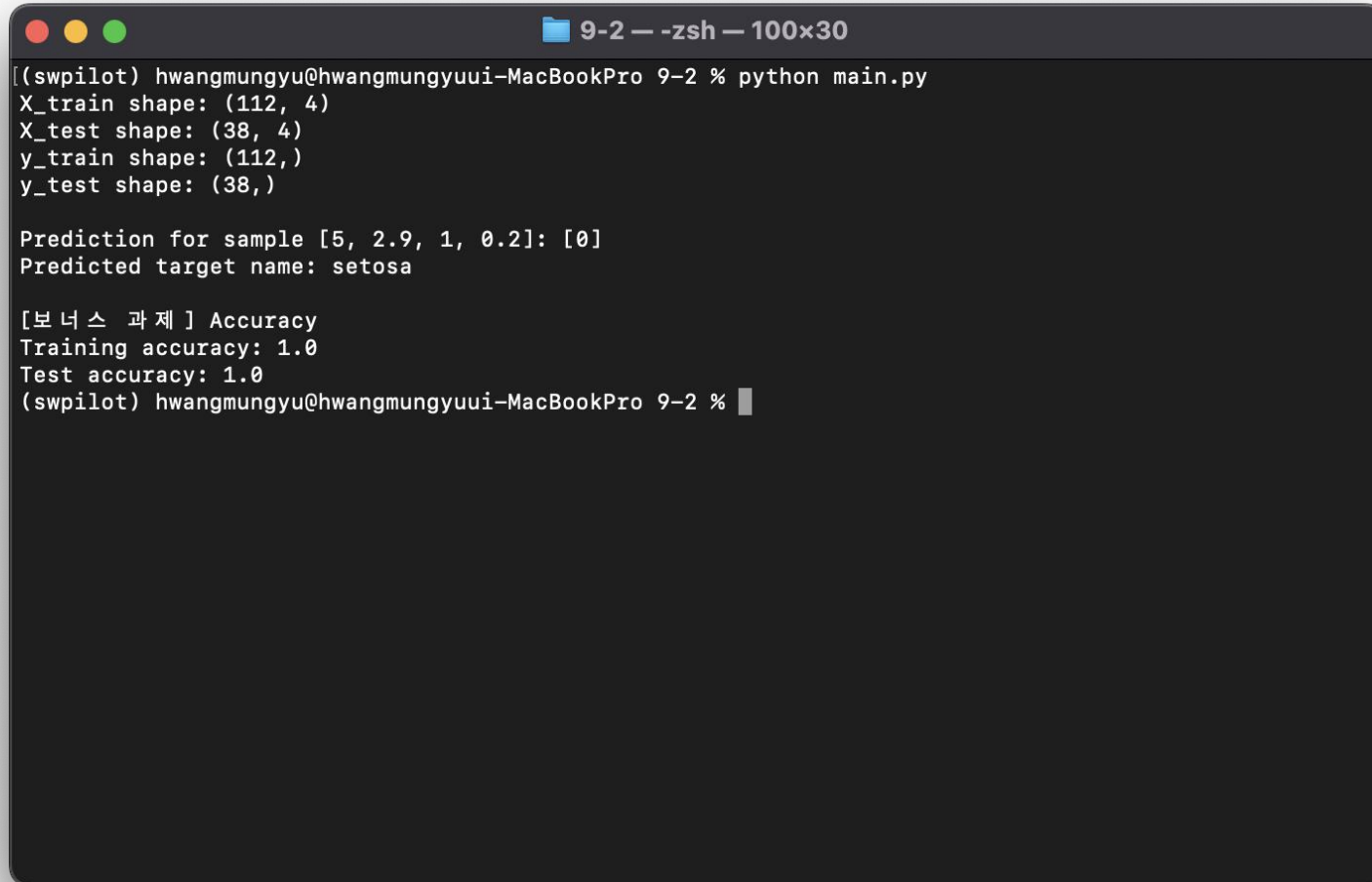


문제 2. 컴퓨터에게 학습을 시켜보자 - 코드

```
1 # 과정 9 - (문제2) 컴퓨터에게 학습을 시켜보자
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.neighbors import KNeighborsClassifier
5
6 def main():
7     # iris 데이터셋 로드
8     iris = load_iris()
9     data = iris['data']
10    target = iris['target']
11
12    # train/test 데이터 분할
13    x_train, x_test, y_train, y_test = train_test_split(
14        data, target, test_size=0.25, random_state=42
15    )
16
17    # 분할된 데이터 정보 출력
18    print('X_train shape:', x_train.shape)
19    print('X_test shape:', x_test.shape)
20    print('y_train shape:', y_train.shape)
21    print('y_test shape:', y_test.shape)
```

```
1 # 모델 학습 (KNN, 이웃 수 = 1)
2 model = KNeighborsClassifier(n_neighbors=1)
3 model.fit(x_train, y_train)
4
5 # 새로운 데이터 예측
6 sample = [[5, 2.9, 1, 0.2]]
7 prediction = model.predict(sample)
8 print('\nPrediction for sample [5, 2.9, 1, 0.2]:', prediction)
9 print('Predicted target name:', iris['target_names'][prediction[0]])
10
11 # [보너스 과제] 모델 평가
12 train_score = model.score(x_train, y_train)
13 test_score = model.score(x_test, y_test)
14 print('\n[보너스 과제] Accuracy')
15 print('Training accuracy:', train_score)
16 print('Test accuracy:', test_score)
17
18
19 if __name__ == '__main__':
20     main()
```

문제 2. 컴퓨터에게 학습을 시켜보자 - 구현화면



```
9-2 --zsh-- 100x30
[swpilot] hwangmungyu@hwangmungyuui-MacBookPro 9-2 % python main.py
X_train shape: (112, 4)
X_test shape: (38, 4)
y_train shape: (112,)
y_test shape: (38,)

Prediction for sample [5, 2.9, 1, 0.2]: [0]
Predicted target name: setosa

[보너스 과제] Accuracy
Training accuracy: 1.0
Test accuracy: 1.0
[swpilot] hwangmungyu@hwangmungyuui-MacBookPro 9-2 %
```

문제 3. 데이터 전처리 Min-Max - 코드

```
1 # 과정 9 - (문제3) 데이터 전처리 Min-Max
2 import pandas as pd
3 from sklearn.preprocessing import MinMaxScaler, StandardScaler # [보너스 과제 포함]
4
5 def load_abalone_data(data_path, attr_path):
6     with open(attr_path, 'r', encoding='utf-8') as f:
7         columns = [line.strip() for line in f if line.strip()]
8
9     df = pd.read_csv(data_path, header=None, names=columns)
10    return df
11
12 def min_max_scaling_manual(df):
13     scaled_df = df.copy()
14     for column in scaled_df.columns:
15         col_min = scaled_df[column].min()
16         col_max = scaled_df[column].max()
17         scaled_df[column] = (scaled_df[column] - col_min) / (col_max - col_min)
18     return scaled_df
19
20 def main():
21     # 데이터 로드
22     data_path = 'src/Process9/9-3/9-3-abalone.txt'
23     attr_path = 'src/Process9/9-3/9-3-abalone_attributes.txt'
24     df = load_abalone_data(data_path, attr_path)
```

```
1 # label 추출 및 Sex 컬럼 제거
2 label = df['Sex']
3 df = df.drop(columns=['Sex'])
4
5 print('원본 데이터 샘플:')
6 print(df.head())
7
8 # 수동 Min-Max Scaling
9 scaled_manual = min_max_scaling_manual(df)
10 print('\n[수동 Min-Max Scaling 결과]')
11 print(scaled_manual.head())
12
13 # Scikit-learn Min-Max Scaling
14 scaler = MinMaxScaler()
15 scaled_auto = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)
16 print('\n[Sklearn Min-Max Scaling 결과]')
17 print(scaled_auto.head())
18
19 # [보너스 과제] Standard Scaling
20 std_scaler = StandardScaler()
21 scaled_standard = pd.DataFrame(std_scaler.fit_transform(df), columns=df.columns)
22 print('\n[보너스 과제] Standard Scaling 결과:')
23 print(scaled_standard.head())
24
25
26 if __name__ == '__main__':
27     main()
```


문제 3. 데이터 전처리 Min-Max - 구현화면

```
swpilotPyhon5 — -zsh — 100x39
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process9/9-3/main.py]
원본 데이터 샘플 :
  Length  Diameter  Height  Whole weight  Shucked weight  Viscera weight  Shell weight  Rings
0   0.455    0.365   0.095    0.5140    0.2245    0.1010    0.150    15
1   0.350    0.265   0.090    0.2255    0.0995    0.0485    0.070     7
2   0.530    0.420   0.135    0.6770    0.2565    0.1415    0.210     9
3   0.440    0.365   0.125    0.5160    0.2155    0.1140    0.155    10
4   0.330    0.255   0.080    0.2050    0.0895    0.0395    0.055     7

[수동 Min-Max Scaling 결과]
  Length  Diameter  Height  ...  Viscera weight  Shell weight  Rings
0  0.513514  0.521008  0.084071  ...    0.132324    0.147982  0.500000
1  0.371622  0.352941  0.079646  ...    0.063199    0.068261  0.214286
2  0.614865  0.613445  0.119469  ...    0.185648    0.207773  0.285714
3  0.493243  0.521008  0.110619  ...    0.149440    0.152965  0.321429
4  0.344595  0.336134  0.070796  ...    0.051350    0.053313  0.214286

[5 rows x 8 columns]

[Sklearn Min-Max Scaling 결과]
  Length  Diameter  Height  ...  Viscera weight  Shell weight  Rings
0  0.513514  0.521008  0.084071  ...    0.132324    0.147982  0.500000
1  0.371622  0.352941  0.079646  ...    0.063199    0.068261  0.214286
2  0.614865  0.613445  0.119469  ...    0.185648    0.207773  0.285714
3  0.493243  0.521008  0.110619  ...    0.149440    0.152965  0.321429
4  0.344595  0.336134  0.070796  ...    0.051350    0.053313  0.214286

[5 rows x 8 columns]

[보너스 과제] Standard Scaling 결과 :
  Length  Diameter  Height  ...  Viscera weight  Shell weight  Rings
0 -0.574558 -0.432149 -1.064424  ...   -0.726212   -0.638217  1.571544
1 -1.448986 -1.439929 -1.183978  ...   -1.205221   -1.212987 -0.910013
2  0.050033  0.122130 -0.107991  ...   -0.356690   -0.207139 -0.289624
3 -0.699476 -0.432149 -0.347099  ...   -0.607600   -0.602294  0.020571
4 -1.615544 -1.540707 -1.423087  ...   -1.287337   -1.320757 -0.910013

[5 rows x 8 columns]
(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```

문제 4. 데이터 전처리 Sampling - 코드

```
1 # 과정9 - (문제4) 데이터 전처리 Sampling
2 import pandas as pd
3 from collections import Counter
4 from sklearn.preprocessing import LabelEncoder
5 from sklearn.utils import resample
6 from imblearn.over_sampling import SMOTE # [보너스 과제]
7
8 def load_abalone_data(data_path, attr_path):
9     with open(attr_path, 'r', encoding='utf-8') as f:
10         columns = [line.strip() for line in f if line.strip()]
11
12     df = pd.read_csv(data_path, header=None, names=columns)
13     return df
14
15 def random_over_sampling(data, label):
16     df = data.copy()
17     df['label'] = label
18     max_count = df['label'].value_counts().max()
19     resampled_list = []
20
21     for value, group in df.groupby('label'):
22         if len(group) < max_count:
23             upsampled = resample(group, replace=True, n_samples=max_count, random_state=42)
24         else:
25             upsampled = group
26         resampled_list.append(upsampled)
27
28     result = pd.concat(resampled_list).sample(frac=1, random_state=42).reset_index(drop=True)
29     return result.drop(columns=['label']), result['label']
30
31 def random_under_sampling(data, label):
32     df = data.copy()
33     df['label'] = label
34     min_count = df['label'].value_counts().min()
35     resampled_list = []
36
37     for value, group in df.groupby('label'):
38         downsampled = resample(group, replace=False, n_samples=min_count, random_state=42)
39         resampled_list.append(downsampled)
40
41     result = pd.concat(resampled_list).sample(frac=1, random_state=42).reset_index(drop=True)
42     return result.drop(columns=['label']), result['label']
43
44 def apply_smote(data, label):
45     smote = SMOTE(random_state=42)
46     return smote.fit_resample(data, label)
```

```
1 def print_label_distribution(label, title):
2     print(f'\n{title} 분포:')
3     counts = Counter(label)
4     for k, v in counts.items():
5         print(f'{k}: {v}')
6
7 def main():
8     data_path = 'src/Process9/9-4/9-4-abalone.txt'
9     attr_path = 'src/Process9/9-4/9-4-abalone_attributes.txt'
10
11     # 데이터 로드
12     df = load_abalone_data(data_path, attr_path)
13
14     # label: 성별, data: 수치형 정보
15     label = df['Sex']
16     data = df.drop(columns=['Sex'])
17
18     # label을 숫자로 인코딩 (SMOTE를 위해 필요)
19     le = LabelEncoder()
20     encoded_label = le.fit_transform(label)
21
22     # 원본 분포
23     print_label_distribution(label, '원본')
24
25     # Random Over Sampling
26     over_data, over_label = random_over_sampling(data, label)
27     print_label_distribution(over_label, 'Random Over Sampling')
28
29     # Random Under Sampling
30     under_data, under_label = random_under_sampling(data, label)
31     print_label_distribution(under_label, 'Random Under Sampling')
32
33     # [보너스 과제] SMOTE
34     smote_data, smote_label = apply_smote(data, encoded_label)
35     decoded_smote_label = le.inverse_transform(smote_label)
36     print_label_distribution(decoded_smote_label, '[보너스 과제] SMOTE 결과')
37
38 if __name__ == '__main__':
39     main()
```


문제 4. 데이터 전처리 Sampling - 구현화면

```
swpilotPyhon5 — -zsh — 100x39
[(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process9/9-4/main.py ]

원본 분포 :
M: 1528
F: 1307
I: 1342

Random Over Sampling 분포 :
F: 1528
M: 1528
I: 1528

Random Under Sampling 분포 :
F: 1307
M: 1307
I: 1307

[보너스 과제] SMOTE 결과 분포 :
M: 1528
F: 1528
I: 1528
(swpilot) hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```

문제 5. 차원의 저주를 풀어라 - 코드

```
1 # 과정 9 - (문제5) 차원의 저주를 풀어라
2 from sklearn.datasets import load_digits
3 from sklearn.decomposition import PCA
4 import matplotlib.pyplot as plt # [보너스 과제]
5 import numpy as np
6
7 def main():
8     # 데이터셋 불러오기
9     digits = load_digits()
10
11     # DESCR 항목 출력
12     print('DESCR:')
13     print(digits['DESCR'])
14
15     # data, label 추출
16     data = digits['data']
17     label = digits['target']
18
19     print('\nData shape:', data.shape)
20     print('Label shape:', label.shape)
21
22     # 첫 번째 샘플을 8x8 행렬로 변환하여 시각 확인
23     first_image = data[0].reshape((8, 8))
24
25     plt.figure(figsize=(3, 3))
26     plt.imshow(first_image, cmap='gray')
27     plt.title(f'Digit: {label[0]}')
28     plt.axis('off')
29     plt.tight_layout()
30     plt.show()
```

```
1 # PCA를 통한 2차원 축소
2 pca = PCA(n_components=2)
3 data_pca = pca.fit_transform(data)
4
5 print('\nPCA 결과 shape:', data_pca.shape)
6
7 # [보너스 과제] 2차원 시각화
8 plt.figure(figsize=(8, 6))
9 for digit in np.unique(label):
10     idx = label == digit
11     plt.scatter(data_pca[idx, 0], data_pca[idx, 1], label=str(digit), alpha=0.7)
12
13 plt.title('Digits PCA (2D)')
14 plt.xlabel('Principal Component 1')
15 plt.ylabel('Principal Component 2')
16 plt.legend(title='Digit')
17 plt.grid(False) # 격자 제거
18 plt.tight_layout()
19 plt.show()
20
21
22 if __name__ == '__main__':
23     main()
```

문제 5. 차원의 저주를 풀어라 - 구현화면

```
swpilotPython5 — python src/Process9/9-5/main.py — 100x39

This is a copy of the test set of the UCI ML hand-written digits datasets
https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits

The data set contains images of hand-written digits: 10 classes where
each class refers to a digit.

Preprocessing programs made available by NIST were used to extract
normalized bitmaps of handwritten digits from a preprinted form. From a
total of 43 people, 30 contributed to the training set and different 13
to the test set. 32x32 bitmaps are divided into nonoverlapping blocks of
4x4 and the number of on pixels are counted in each block. This generates
an input matrix of 8x8 where each element is an integer in the range
0..16. This reduces dimensionality and gives invariance to small
distortions.

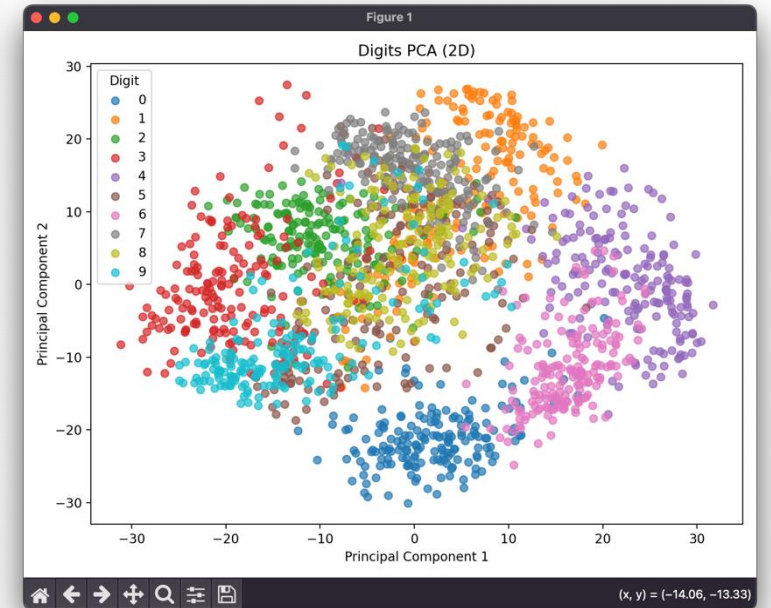
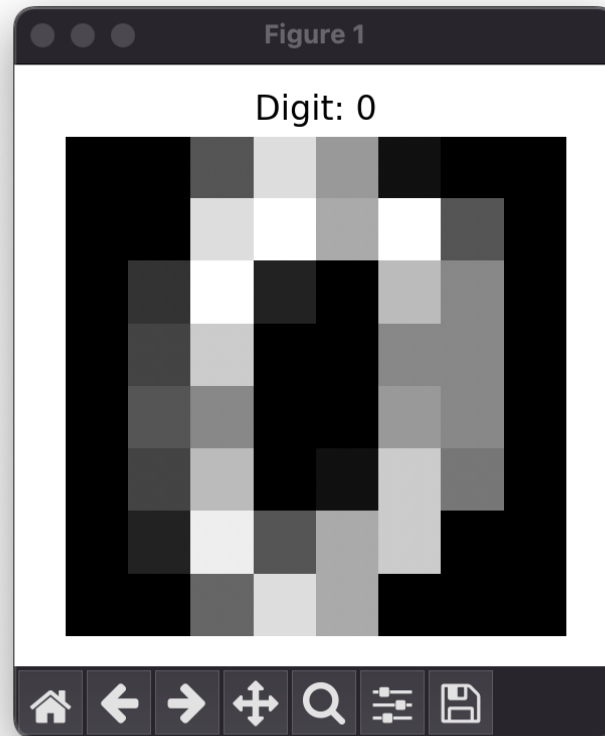
For info on NIST preprocessing routines, see M. D. Garris, J. L. Blue, G.
T. Candela, D. L. Dimmick, J. Geist, P. J. Grother, S. A. Janet, and C.
L. Wilson, NIST Form-Based Handprint Recognition System, NISTIR 5469,
1994.

.. dropdown:: References

- C. Kaynak (1995) Methods of Combining Multiple Classifiers and Their
  Applications to Handwritten Digit Recognition, MSc Thesis, Institute of
  Graduate Studies in Science and Engineering, Bogazici University.
- E. Alpaydin, C. Kaynak (1998) Cascading Classifiers, Kybernetika.
- Ken Tang and Ponnuthurai N. Suganthan and Xi Yao and A. Kai Qin.
  Linear dimensionality reduction using relevance weighted LDA. School of
  Electrical and Electronic Engineering Nanyang Technological University.
  2005.
- Claudio Gentile. A New Approximate Maximal Margin Classification
  Algorithm. NIPS. 2000.

Data shape: (1797, 64)
Label shape: (1797,)

PCA 결과 shape: (1797, 2)
```



문제 6. 분명하게 말해 두기 - 코드

```
1 # 과정 9 - (문제6) 분명하게 말해 두기
2 import pandas as pd
3 from sklearn.preprocessing import LabelEncoder, OneHotEncoder
4 import numpy as np
5
6 def load_abalone_data(data_path, attr_path):
7     with open(attr_path, 'r', encoding='utf-8') as f:
8         columns = [line.strip() for line in f if line.strip()]
9
10    df = pd.read_csv(data_path, header=None, names=columns)
11    return df
12
13 def label_encode(label_series):
14     encoder = LabelEncoder()
15     encoded = encoder.fit_transform(label_series)
16     print('Label Encoding 결과:', encoded[:10])
17     print('클래스:', list(encoder.classes_))
18     return encoded, encoder
19
20 def one_hot_encode(encoded_labels):
21     encoder = OneHotEncoder(sparse_output=False)
22     encoded_2d = np.array(encoded_labels).reshape(-1, 1)
23     one_hot = encoder.fit_transform(encoded_2d)
24     print('\nOne-Hot Encoding 결과 (앞 5개):\n', one_hot[:5])
25     print('카테고리:', encoder.categories_)
26     return one_hot
27
28 def handle_missing_values(df):
29     print('\n[보너스 과제] 결측값 처리 예시')
30     df_with_nan = df.copy()
31     df_with_nan.iloc[0, 1] = np.nan # 임의 결측치 삽입
32     print('결측값 전:\n', df_with_nan.head())
33     df_filled = df_with_nan.fillna(df_with_nan.mean(numeric_only=True))
34     print('결측값 처리 후:\n', df_filled.head())
35
36 def handle_noise(df):
37     print('\n[보너스 과제] 노이즈 처리 예시')
38     df_noisy = df.copy()
39     df_noisy.iloc[0, 2] = 9999 # 임의 노이즈 삽입
40     print('노이즈 전:\n', df_noisy.head())
41     # 단순 평균 대체
42     col_mean = df_noisy[df_noisy.iloc[:, 2] < 1000].iloc[:, 2].mean()
43     df_noisy.iloc[0, 2] = col_mean
44     print('노이즈 처리 후:\n', df_noisy.head())
```

```
1 def handle_outliers(df):
2     print('\n[보너스 과제] 이상값 처리 예시')
3     df_outlier = df.copy()
4     col = df_outlier.iloc[:, 3]
5     q1 = col.quantile(0.25)
6     q3 = col.quantile(0.75)
7     iqr = q3 - q1
8     lower_bound = q1 - 1.5 * iqr
9     upper_bound = q3 + 1.5 * iqr
10    outliers = (col < lower_bound) | (col > upper_bound)
11    print('이상값 개수:', outliers.sum())
12    df_outlier.loc[outliers, col.name] = col.median()
13    print('이상값 처리 후 일부:\n', df_outlier[outliers].head())
14
15 def main():
16     data_path = 'src/Process9/9-4/9-4-abalone.txt'
17     attr_path = 'src/Process9/9-4/9-4-abalone_attributes.txt'
18
19     df = load_abalone_data(data_path, attr_path)
20
21     label = df['Sex']
22     data = df.drop(columns=['Sex'])
23
24     # Label Encoding
25     encoded_label, label_encoder = label_encode(label)
26
27     # One-Hot Encoding
28     one_hot_encoded = one_hot_encode(encoded_label)
29
30     # [보너스 과제] 데이터 전처리 예시
31     handle_missing_values(data)
32     handle_noise(data)
33     handle_outliers(data)
34
35 if __name__ == '__main__':
36     main()
```

문제 6. 분명하게 말해 두기 - 구현화면

```
swpilotPython5 --zsh 160x55
([swpilot] hwangmungyu@hwangmungyuui-MacBookPro swpilotPython5 % python src/Process9/9-6/main.py
Label Encoding 결과 : [2 2 0 2 1 1 0 0 2 0]
클래스 : ['F', 'I', 'M']

One-Hot Encoding 결과 (앞 5개):
[[0. 0. 1.]
 [0. 0. 1.]
 [1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]]
카테고리 : [array([0, 1, 2])]

[보너스 과제] 결측값 처리 예시
결측값 전 :
  Length Diameter Height Whole weight Shucked weight Viscera weight Shell weight Rings
0  0.455      NaN   0.095    0.5140      0.2245      0.1010      0.150      15
1  0.350    0.265   0.090    0.2255      0.0995      0.0485      0.070      7
2  0.530    0.420   0.135    0.6770      0.2565      0.1415      0.210      9
3  0.440    0.365   0.125    0.5160      0.2155      0.1140      0.155     10
4  0.330    0.255   0.080    0.2050      0.0895      0.0395      0.055      7
결측값 처리 후 :
  Length Diameter Height Whole weight Shucked weight Viscera weight Shell weight Rings
0  0.455  0.407892   0.095    0.5140      0.2245      0.1010      0.150      15
1  0.350  0.265000   0.090    0.2255      0.0995      0.0485      0.070      7
2  0.530  0.420000   0.135    0.6770      0.2565      0.1415      0.210      9
3  0.440  0.365000   0.125    0.5160      0.2155      0.1140      0.155     10
4  0.330  0.255000   0.080    0.2050      0.0895      0.0395      0.055      7

[보너스 과제] 노이즈 처리 예시
노이즈 전 :
  Length Diameter Height Whole weight Shucked weight Viscera weight Shell weight Rings
0  0.455    0.365 9999.000    0.5140      0.2245      0.1010      0.150      15
1  0.350    0.265   0.090    0.2255      0.0995      0.0485      0.070      7
2  0.530    0.420   0.135    0.6770      0.2565      0.1415      0.210      9
3  0.440    0.365   0.125    0.5160      0.2155      0.1140      0.155     10
4  0.330    0.255   0.080    0.2050      0.0895      0.0395      0.055      7
노이즈 처리 후 :
  Length Diameter Height Whole weight Shucked weight Viscera weight Shell weight Rings
0  0.455    0.365 0.139527    0.5140      0.2245      0.1010      0.150      15
1  0.350    0.265 0.090000    0.2255      0.0995      0.0485      0.070      7
2  0.530    0.420 0.135000    0.6770      0.2565      0.1415      0.210      9
3  0.440    0.365 0.125000    0.5160      0.2155      0.1140      0.155     10
4  0.330    0.255 0.080000    0.2050      0.0895      0.0395      0.055      7

[보너스 과제] 이상값 처리 예시
이상값 개수 : 30
이상값 처리 후 일부 :
  Length Diameter Height Whole weight Shucked weight Viscera weight Shell weight Rings
165  0.725    0.570   0.190    0.7995      1.0705      0.483      0.7250     14
358  0.745    0.585   0.215    0.7995      0.9265      0.472      0.7000     17
891  0.730    0.595   0.230    0.7995      1.1465      0.419      0.8970     17
1051 0.735    0.600   0.220    0.7995      1.1335      0.440      0.6000     11
1052 0.765    0.600   0.220    0.7995      1.0070      0.509      0.6205     12

([swpilot] hwangmungyu@hwangmungyuui-MacBookPro swpilotPython5 %
```


문제 7. 맛있는 토마토 찾기 - 코드

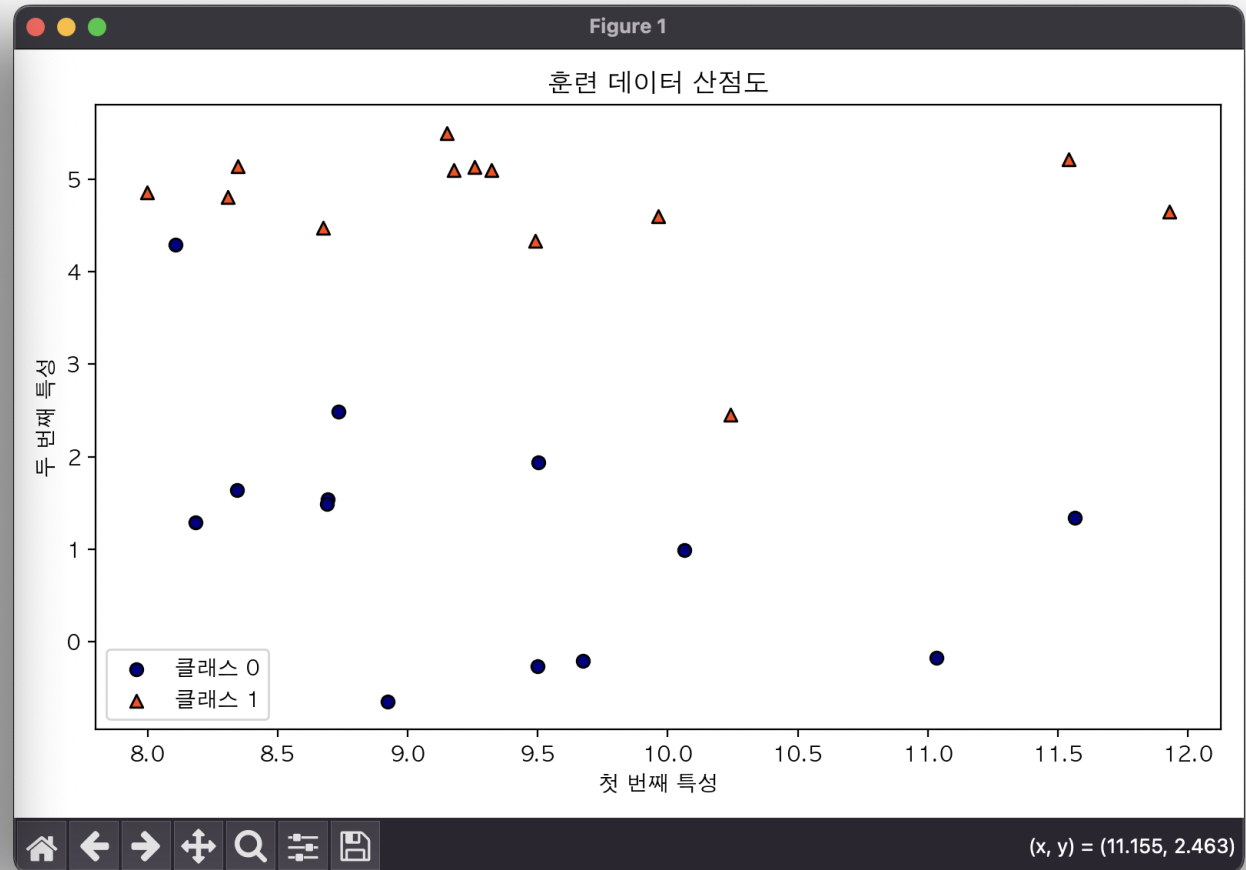
```
1 # 과정 9 - (문제7) 맛있는 토마토 찾기
2 import matplotlib.pyplot as plt
3 import numpy as np
4 from sklearn.model_selection import train_test_split
5 from sklearn.neighbors import KNeighborsClassifier
6 import mglearn
7 import matplotlib
8
9 matplotlib.rc('font', family='AppleGothic')
10 plt.rcParams['axes.unicode_minus'] = False
11
12 def plot_scatter(x, y, title='산점도 시각화'):
13     plt.figure(figsize=(8, 5))
14     for label, marker, color in zip([0, 1], ['o', '^'], ['navy', 'orangered']):
15         plt.scatter(
16             x[y == label, 0], x[y == label, 1],
17             marker=marker, color=color, label=f'클래스 {label}', edgecolor='black'
18         )
19     plt.xlabel('첫 번째 특성')
20     plt.ylabel('두 번째 특성')
21     plt.legend()
22     plt.title(title)
23     plt.tight_layout()
24     plt.show()
```

```
1 def main():
2     # 데이터 생성
3     x, y = mglearn.datasets.make_forge()
4
5     # 산점도 시각화
6     plot_scatter(x, y, '훈련 데이터 산점도')
7
8     # 데이터 분할
9     x_train, x_test, y_train, y_test = train_test_split(
10         x, y, random_state=0
11     )
12
13     # KNN 학습 및 평가 (n_neighbors=3)
14     model = KNeighborsClassifier(n_neighbors=3)
15     model.fit(x_train, y_train)
16     accuracy = model.score(x_test, y_test)
17     print('기본 모델 정확도 (n_neighbors=3):', accuracy)
18
19     # [보너스 과제] n_neighbors = 1 ~ 9 변화에 따른 정확도
20     print('\n[보너스 과제] n_neighbors 변화에 따른 테스트 정확도:')
21     for k in range(1, 10):
22         knn = KNeighborsClassifier(n_neighbors=k)
23         knn.fit(x_train, y_train)
24         acc = knn.score(x_test, y_test)
25         print(f'n_neighbors = {k} → 정확도: {acc:.2f}')
26
27
28 if __name__ == '__main__':
29     main()
```


문제 7. 맛있는 토마토 찾기 - 구현화면

```
swpilotPyhon5 — -zsh — 99x33
[swpilot] hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 % python src/Process9/9-7/main.py
기본 모델 정확도 (n_neighbors=3): 0.8571428571428571

[보너스 과제] n_neighbors 변화에 따른 테스트 정확도 :
n_neighbors = 1 → 정확도 : 0.86
n_neighbors = 2 → 정확도 : 0.86
n_neighbors = 3 → 정확도 : 0.86
n_neighbors = 4 → 정확도 : 0.86
n_neighbors = 5 → 정확도 : 0.86
n_neighbors = 6 → 정확도 : 0.86
n_neighbors = 7 → 정확도 : 0.86
n_neighbors = 8 → 정확도 : 0.86
n_neighbors = 9 → 정확도 : 0.86
[swpilot] hwangmungyu@hwangmungyuui-MacBookPro swpilotPyhon5 %
```



문제 8. 진짜 맛있는 토마토 찾기 - 코드

```
1 # 과정 9 - (문제8) 진짜 맛있는 토마토 찾기
2 import matplotlib.pyplot as plt
3 from mglearn.datasets import make_forge
4 from sklearn.model_selection import train_test_split
5 from sklearn.neighbors import KNeighborsClassifier
6 import mglearn
7 import numpy as np
8 import matplotlib
9
10 matplotlib.rc('font', family='AppleGothic')
11 plt.rcParams['axes.unicode_minus'] = False
12
13 def plot_decision_boundary(model, x, y, title):
14     mglearn.plots.plot_2d_separator(model, x, fill=True, alpha=0.4)
15     mglearn.discrete_scatter(x[:, 0], x[:, 1], y)
16     plt.title(title)
17     plt.xlabel('첫 번째 특성')
18     plt.ylabel('두 번째 특성')
19     plt.tight_layout()
20     plt.show()
21
22 def main():
23     # 데이터 생성 및 분리
24     x, y = mglearn.datasets.make_forge()
25     x_train, x_test, y_train, y_test = train_test_split(
26         x, y, random_state=0
27     )
28
29     training_accuracy = []
30     test_accuracy = []
31     neighbor_settings = range(1, 11)
```

```
1     for n_neighbors in neighbor_settings:
2         model = KNeighborsClassifier(n_neighbors=n_neighbors)
3         model.fit(x_train, y_train)
4         train_acc = model.score(x_train, y_train)
5         test_acc = model.score(x_test, y_test)
6         training_accuracy.append(train_acc)
7         test_accuracy.append(test_acc)
8
9     # 꺾은선 그래프 출력
10    plt.figure(figsize=(8, 5))
11    plt.plot(neighbor_settings, training_accuracy, label='훈련 정확도')
12    plt.plot(neighbor_settings, test_accuracy, label='테스트 정확도')
13    plt.xlabel('n_neighbors')
14    plt.ylabel('정확도 (accuracy)')
15    plt.title('KNN 정확도 비교')
16    plt.legend()
17    plt.ylim(0.5, 1)
18    plt.tight_layout()
19    plt.grid(False)
20    plt.show()
21
22    # [보너스 과제] 결정 경계 시각화 (n = 1, 3, 9)
23    for k in [1, 3, 9]:
24        model = KNeighborsClassifier(n_neighbors=k)
25        model.fit(x, y)
26        plot_decision_boundary(model, x, y, f'결정 경계 (n_neighbors = {k})')
27
28 if __name__ == '__main__':
29     main()
```

문제 8. 진짜 맛있는 토마토 찾기 - 구현화면

